GROUP 1

Image Classification

Using Convolutional Neural Networks

CIFAR-10 Dataset · Deep Learning · Transfer Learning

Team Members

- Charleson
- Harn
- João

OVERVIEW

Executive Summary

Goal

Build a CNN to classify images from CIFAR-10 into 10 classes with high accuracy

Approach

Progressed from a simple CNN to a deep VGG-style model, then added Transfer Learning

Key Techniques

BatchNormalization, Dropout, Data Augmentation, EarlyStopping, Transfer Learning (MobileNetV2)

Best Result

Final model (MobileNetV2) achieved 90.21% Test Accuracy

DATASET

CIFAR-10 Dataset

60,000	10	32×32	50K/10K
Total Images	Classes	Image Size	Train / Test Split

The 10 Classes:

✈ Airplane	🚗 Automobile	🐦 Bird	🐱 Cat	🦌 Deer
🐶 Dog	🐸 Frog	🐴 Horse	🚢 Ship	🚚 Truck

PREPROCESSING

Data Preprocessing

Load Data

50K train
10K test

Normalize

Pixels
÷255 → [0,1]

One-Hot Encode

3 → [0,0,0,1,
0,0,0,0,0,0]

Train/Val Split

90% train
10% val

Data Augmentation

Flip, Rotate,
Zoom, Shift

💡 *Data Augmentation applies random rotations, horizontal flips, shifts and zoom to artificially increase training diversity — reducing overfitting.*

ARCHITECTURE

Model Progression — Key Milestones

10 models built in total — showing the 4 most significant steps

Model 1

Baseline

Test: 73.56%

- 3 Conv layers, No BN
- No Dropout, No Aug
- Adam · LR=0.001 · 10 epochs
- Overfitting: Train 85% vs Test 73%

Model 3

+ Dropout

Test: 78.45%

- Added Dropout
- Bach size raised to 128
- Train≈Val accuracy
- Overfitting reduced

Model 10

+ BN + Aug + L2

Test: 85.15%

- BatchNorm + Dropout
- Data Augmentation
- L2 Regularizer
- LR Scheduler · 50 epochs

Model 8

Transfer Learning + Data Augmentation

Test: 90.21%

- MobileNetV2 pretrained
- Global Avg Pooling
- 2-Phase Fine-Tuning
- LR Scheduler + EarlyStopping

OPTIMIZATION

Optimization Techniques

OPT

Adam Optimizer

LR = 0.0005. Adapts learning rate per parameter — faster and more stable than SGD.

BN

BatchNormalization

Normalizes layer outputs. Stabilizes training, allows higher learning rates.

DO

Dropout

25% after each block, 50% before output. Randomly disables neurons to prevent overfitting.

AUG

Data Augmentation

Rotation, flip, shift, zoom. Creates image variations each epoch for better generalization.

ES

EarlyStopping

Patience = 15 epochs. Stops training when val_loss stops improving, restores best weights.

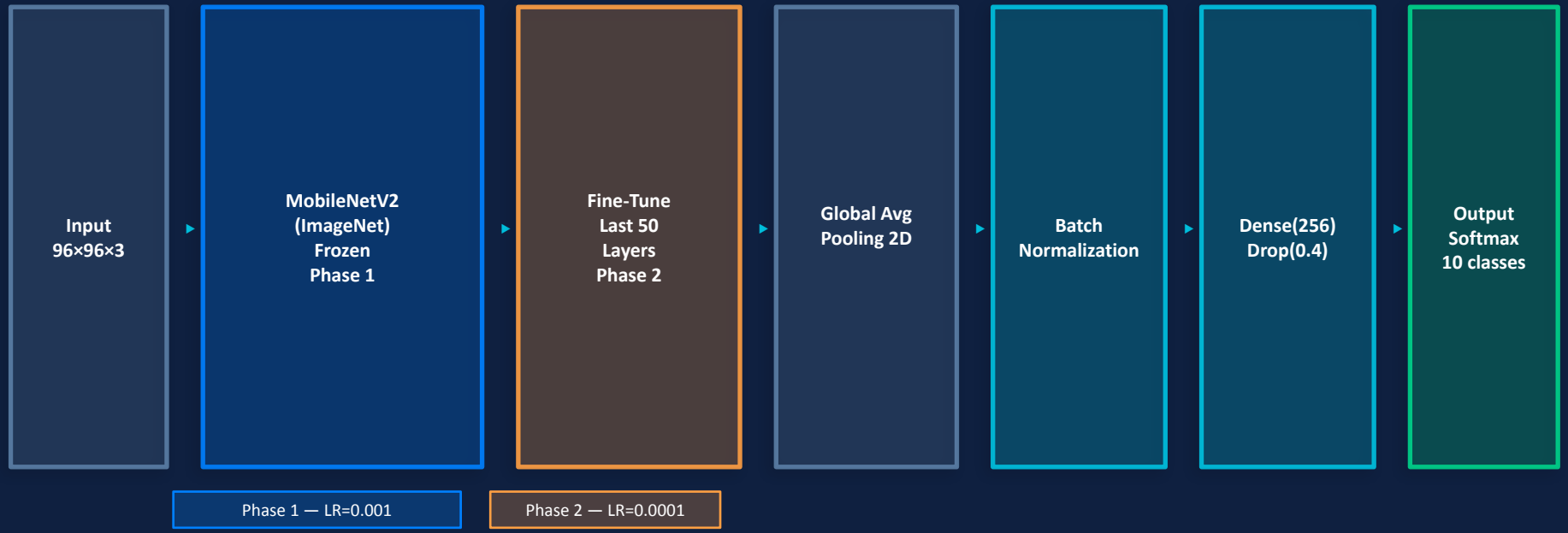
LR

ReduceLROnPlateau

Patience = 7 epochs. Halves learning rate when model is stuck near the optimum.

ARCHITECTURE

Final CNN Architecture — Model 8



Pre-trained on ImageNet · 1.2M images — knows edges, shapes, textures

include_top=False · Remove original classifier, add our own

Resize to 96x96 · CIFAR-10 is 32×32 — too small for MobileNetV2

Transfer Learning -MobileNetV2

Why MobileNetV2?

- Pre-trained on 1.2M ImageNet images
- Already knows edges, textures & shapes
- Lightweight and efficient on GPU
- include_top=False ⟶ custom classifier
- Input resized to 96×96

TRANSFER LEARNING

MobileNetV2 with Data augmentation

Transfer Learning

Phase 1 — Train Classifier

Base model frozen · LR = 0.001 · 20 epochs
Only Dense layers updated

Phase 2 — Fine-Tuning

Last 50 layers unfrozen
LR = 0.0001 (very low)
50 epochs · Callbacks active

Data Augmentation

Horizontal flip = True

Flip entire image left/right

Width & Height Shift = 0.1

Shifts image horizontally & vertically up to 10%

Rotation range = 15

Rotates image up to 15º

Zoom range = 0.1

Zooms in/out affecting both width and height simultaneously

Result: ~90%+ Test Accuracy

EVALUATION

Model Evaluation & Results

Model	Key Changes	Test Accuracy	Overfitting
Model 1 — Baseline	No regularization · Adam · 10 epochs	**73.56%**	Severe
Model 3 — + Dropout	Dropout ·Batch Size (128)	**77.87%**	Low
Model 10 — + BN + Aug + L2	BatchNorm · Data Aug · L2 · LR Scheduler · 50 ep	**85.15%**	None
Model 8 — TL + Aug (FINAL)	MobileNetV2 ·Data Augmentation Fine-Tuning · LR Scheduler	**90.21%**	None

📊 Evaluation metrics include: Confusion Matrix · Precision · Recall · F1-Score per class · Misclassified image analysis

DEPLOYMENT

Model Deployment

01

Save Model

Saved as .keras format
to Google Drive
model.save(path)

02

Load Model

Reloaded in any session
without retraining
load_model(path)

03

Predict

Resize ⟶ Normalize
Expand dims ⟶ Predict
np.argmax(probs)

04

Custom Image

Upload any photo
Top-3 predictions
Live demo ready

🎬 Live Demo Available — predict any uploaded image in real-time using the trained model

Challenges

01	Problems with the RAM and GPU when running
02	choosing the right hyperparameters, without the model overfiting
03	Cell execution order in Jupyter
04	Google Colab does not support image > 96x96. Resizing: Transfer Learning.

Project Recap & Key Learnings

01	CNN architecture depth directly impacts accuracy — 2 Conv per block beats 1
02	BatchNormalization is the single most impactful regularization technique
03	Data Augmentation effectively reduces overfitting without extra data
04	Transfer Learning with MobileNetV2 achieves superior results with less training
05	Callbacks (EarlyStopping, ReduceLROnPlateau) automate training optimization

Thank You · Questions?